

CTF 5: CBC Padding Oracle Attack

Secure Password Manager Project - CTF Challenge

Rana Elgharabawy | 2026

1. Challenge Overview

This CTF challenge involves exploiting a CBC (Cipher Block Chaining) Padding Oracle vulnerability to decrypt an AES-CBC ciphertext without knowing the secret key. The target server exposes a padding oracle endpoint that reveals whether a submitted ciphertext has valid PKCS#7 padding after decryption. This single bit of information is enough to recover the full plaintext byte by byte.

Property	Value
Challenge	CTF 5 - CBC Padding Oracle
Target URL	http://cbc-ctf.westeurope.azurecontainer.io:5000
Oracle endpoint	POST /oracle
Cipher	AES-128-CBC with PKCS#7 padding
Block size	16 bytes (128 bits)
Goal	Recover the plaintext flag from the ciphertext
Script	CTF/ctf5_new.py

2. Background: AES-CBC Mode

2.1 Encryption

In CBC mode, each plaintext block is XORed with the previous ciphertext block before being encrypted. The first block uses the IV (Initialization Vector) in place of a previous ciphertext block.

```
C[i] = AES_Encrypt( P[i] XOR C[i-1] )  
C[0] = AES_Encrypt( P[0] XOR IV )
```

2.2 Decryption

Decryption reverses the process: each ciphertext block is AES-decrypted to produce an intermediate value, then XORed with the previous ciphertext block to recover the plaintext.

```
P[i] = AES_Decrypt( C[i] ) XOR C[i-1]  
P[0] = AES_Decrypt( C[0] ) XOR IV
```

2.3 PKCS#7 Padding

AES-CBC requires all plaintext blocks to be exactly 16 bytes. PKCS#7 padding appends N bytes each with value N to make the total length a multiple of 16. For example, if 3 padding bytes are needed, the bytes 0x03 0x03 0x03 are appended. A full padding block (16 x 0x10) is added when the message is already a multiple of 16 bytes.

```
Plaintext: 'Hello World' (11 bytes)  
Padded:    'Hello World\x05\x05\x05\x05' (16 bytes)
```

Plaintext: 'ExactlyFifteen!' (15 bytes)
 Padded: 'ExactlyFifteen!\x01' (16 bytes)

3. The Padding Oracle Attack

3.1 What is a Padding Oracle?

A padding oracle is any system that reveals, directly or indirectly, whether a given ciphertext decrypts to a value with valid PKCS#7 padding. This could be a different HTTP status code, a different error message, or a boolean field in a JSON response (as in this challenge: {"valid_padding": true/false}).

3.2 Core Insight

During CBC decryption, the server computes:

```
Intermediate[i] = AES_Decrypt( C[i] )
Plaintext[i]    = Intermediate[i] XOR C[i-1]
```

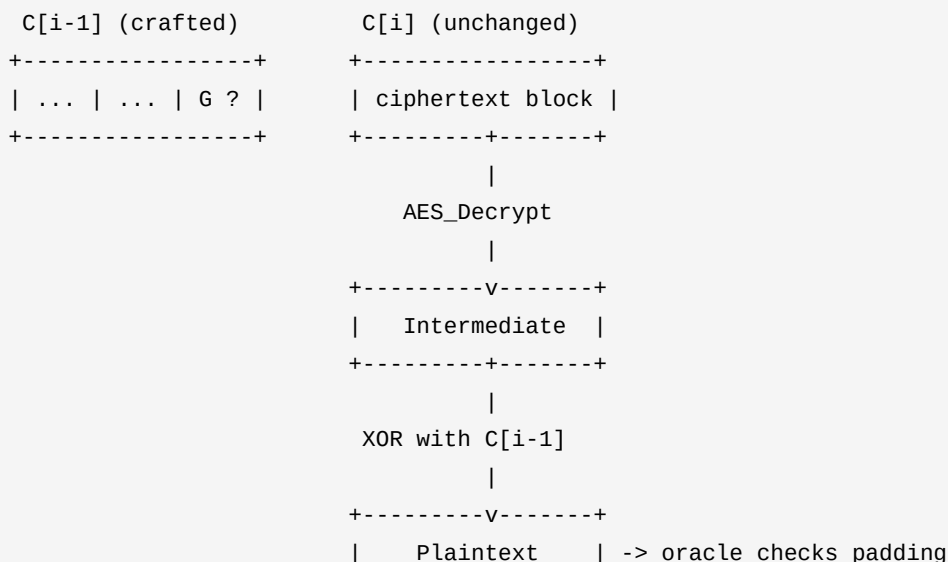
If an attacker replaces $C[i-1]$ with a crafted block, the resulting plaintext changes without altering the Intermediate value (which depends only on $C[i]$ and the secret key). By iterating over all 256 possible byte values and checking the oracle response, the attacker can determine $Intermediate[i]$ and then recover $Plaintext[i]$.

3.3 Byte-by-Byte Recovery

The attack recovers plaintext one byte at a time, working from the last byte to the first within each block:

1. Target the last byte (byte 15). Craft a modified $C[i-1]$ block where all other bytes are arbitrary and byte 15 is guessed from 0 to 255.
2. Submit each guess to the oracle. When `valid_padding` is True, the decrypted last byte equals 0x01, so: guess $XOR\ Intermediate[15] = 0x01$, giving $Intermediate[15] = guess\ XOR\ 0x01$.
3. Recover the true plaintext byte: $Plaintext[15] = Intermediate[15]\ XOR\ original_C[i-1][15]$.
4. Move to byte 14. Now set the crafted byte 15 = $Intermediate[15]\ XOR\ 0x02$ to produce valid 0x02 0x02 padding, and guess byte 14 the same way.
5. Repeat until all 16 bytes of the block are recovered. Then move to the next block, using the recovered block as the known $C[i-1]$.

3.4 Visual Diagram



+-----+

4. Script Walkthrough (ctf5_new.py)

4.1 Session Setup & Ciphertext Extraction

The script opens an HTTP session with automatic retries (3 attempts, exponential backoff) and fetches the challenge homepage. It then scans the HTML for hex strings whose length is a multiple of 32 characters (i.e., a multiple of 16 bytes) to identify the ciphertext.

```
session = requests.Session()
session.mount("http://", HTTPAdapter(max_retries=Retry(total=3, backoff_factor=0.3)))

page = session.get(Base_URL + "/", timeout=15)

candidates = sorted(
    set(re.findall(r'[0-9a-f]{32,}', page.text)),
    key=len, reverse=True
)
CIPHERTEXT_HEX = next((c for c in candidates if len(c) % 32 == 0), None)
```

4.2 Oracle Format Discovery

The oracle endpoint may accept input as JSON or as form-data, and may expect the ciphertext under different field names. The script probes all combinations automatically, trying field names: 'ciphertext', 'ct', 'hex', 'ciphertext_hex', 'cipher', 'data' - each as both form-data and JSON - until it gets {"valid_padding": true} back from the server.

```
for field in ["ciphertext", "ct", "hex", "ciphertext_hex", "cipher", "data"]:
    for use_json in [False, True]:
        kw = {"json" if use_json else "data": {field: CIPHERTEXT_HEX}}
        r = session.post(Base_URL + "/oracle", timeout=10, **kw)
        if r.json().get("valid_padding") is True:
            USE_JSON, FIELD = use_json, field
            break
```

4.3 Oracle Function

The oracle() function sends a crafted ciphertext to the server and returns the boolean valid_padding response. It retries indefinitely on network errors to handle transient failures during the hundreds of requests the attack requires.

```
def oracle(ct_hex):
    while True:
        try:
            kw = {"json" if USE_JSON else "data": {FIELD: ct_hex}}
            r = session.post(Base_URL + "/oracle", timeout=15, **kw)
            return r.json()["valid_padding"]
        except Exception as e:
            print(f"    [retry] {e}")
```

4.4 Block Parsing

The ciphertext hex string is split into 32-character chunks (16 bytes each). The first block is the IV; the rest are the actual ciphertext blocks to decrypt.

```
blocks = [CIPHERTEXT_HEX[i:i+32] for i in range(0, len(CIPHERTEXT_HEX), 32)]
IV = blocks[0]
```

```
cipher_blocks = blocks[1:]
```

4.5 decrypt_block() - Core Attack Logic

This function recovers all 16 plaintext bytes of a single ciphertext block. It works backwards from byte 15 to byte 0, using the padding oracle to determine each intermediate byte and XOR it against the actual previous ciphertext block to get the plaintext.

```
def decrypt_block(prev_hex, target_hex, label=""):
    intermediate = bytearray(16)          # stores AES_Decrypt(C[i]) values
    for byte_idx in range(15, -1, -1):    # right to left
        padding_val = 16 - byte_idx       # e.g. 1 for byte 15, 2 for byte 14
        crafted = bytearray(16)
        # fix already-known bytes to produce correct padding suffix
        for k in range(byte_idx + 1, 16):
            crafted[k] = intermediate[k] ^ padding_val
        # guess the current byte
        for guess in range(256):
            crafted[byte_idx] = guess
            if oracle(IV + crafted.hex() + target_hex):
                intermediate[byte_idx] = guess ^ padding_val
                pt = intermediate[byte_idx] ^ bytearray.fromhex(prev_hex)[byte_idx]
                print(f" [{label}] byte {16-byte_idx:2d}/16 pt=0x{pt:02x}")
                break
    prev = bytearray.fromhex(prev_hex)
    return bytearray(intermediate[i] ^ prev[i] for i in range(16))
```

The crafted block structure sent to the oracle for each guess is: IV + crafted_prev_block + target_cipher_block. The IV is prepended because the oracle treats the first block as the IV for decryption of the second block, and the crafted block plays the role of C[i-1] for the target block.

4.6 Main Decryption Loop & Padding Removal

All ciphertext blocks are decrypted in order. Each block uses the previous ciphertext block as 'prev' (or the IV for the first block). After all blocks are decrypted, PKCS#7 padding is stripped by reading the last byte as the padding length.

```
plaintext = bytearray()
prev = IV
for i, blk in enumerate(cipher_blocks):
    plaintext += decrypt_block(prev, blk, f"B{i+1}")
    prev = blk

pad_len = plaintext[-1]
if 1 <= pad_len <= 16:
    plaintext = plaintext[:-pad_len]

print(">>> FLAG:", plaintext.decode("utf-8", errors="replace"))
```

5. Complexity & Oracle Call Count

For each byte recovered, the attacker makes up to 256 oracle queries (one per possible byte value). With 16 bytes per block and N ciphertext blocks, the total worst-case oracle calls are:

$\text{Max queries} = N_{\text{blocks}} \times 16 \text{ bytes} \times 256 \text{ guesses}$

On average, a byte is found after 128 guesses (half of 256), so expected calls are approximately $N \times 16 \times 128$. The script uses a persistent session with retry logic to handle the high volume of HTTP requests reliably.

Factor	Detail
Guesses per byte	Up to 256 (average ~128)
Bytes per block	16
Blocks in ciphertext	Depends on message length
Retry strategy	3 retries, 0.3s exponential backoff + infinite retry on exceptions
Timeout per request	15 seconds

6. Why This Works - The Security Flaw

The fundamental flaw is that the server reveals padding validity as a distinct, queryable signal. This transforms an otherwise secure cipher into one that leaks information one bit at a time. AES-CBC with proper padding is computationally secure against ciphertext-only attacks, but the oracle collapses that security by making the internal decryption state observable.

Mitigations used in secure systems:

- Use authenticated encryption (AES-GCM or ChaCha20-Poly1305) which verifies integrity before decryption - invalid ciphertext is rejected before padding is checked.
- Return identical error responses for padding errors and all other decryption failures (no timing differences, no distinct status codes).
- Validate a MAC/HMAC over the ciphertext before attempting decryption (Encrypt-then-MAC).

7. Sample Output

```
Fetching page HTML...
Ciphertext: a1b2c3d4... (3 blocks)

Probing oracle format...
Working oracle: json field='ciphertext'

Decrypting block 1/2 ...
[B1] byte 1/16 pt=0x43 ('C')
[B1] byte 2/16 pt=0x54 ('T')
...
[B1] byte 16/16 pt=0x7b ('{')

Decrypting block 2/2 ...
[B2] byte 1/16 pt=0x66 ('f')
...
[B2] byte 16/16 pt=0x7d ('}')

>>> FLAG: CTF{padding_oracle_is_deadly}
```

8. Requirements

Secure Password Manager - CTF 5 Documentation

Requirement	Details
Python version	Python 3.8+
Dependencies	requests, urllib3 (pip install requests)
Network access	HTTP access to cbc-ctf.westeurope.azurecontainer.io:5000
Execution directory	Any directory - no local files needed
Run command	python CTF/ctf5_new.py